```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib.gridspec import GridSpec
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import roc_auc_score
from sklearn.cluster import KMeans

# ---- Load & Prepare Data
-------------------------------------------------
df = pd.read_csv("Bank_Churn.csv")

le = LabelEncoder()
df_m = df.copy()
df_m["Gender_enc"] = le.fit_transform(df["Gender"])
df_m["Geo_enc"]    = le.fit_transform(df["Geography"])

features =
["CreditScore","Geo_enc","Gender_enc","Age","Tenure","Balance",

"NumOfProducts","HasCrCard","IsActiveMember","EstimatedSalary"]
X, y = df_m[features], df_m["Exited"]
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2,
random_state=42)

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X_tr, y_tr)
auc   = roc_auc_score(y_te, rf.predict_proba(X_te)[:,1])
acc   = rf.score(X_te, y_te)
fi    = pd.Series(rf.feature_importances_,
index=features).sort_values()

feat_labels = {"CreditScore":"Credit
score","Geo_enc":"Geography","Gender_enc":"Gender",
               "Age":"Age","Tenure":"Tenure","Balance":"Balance",
               "NumOfProducts":"# products","HasCrCard":"Has cr.
card",
               "IsActiveMember":"Active
member","EstimatedSalary":"Est. salary"}

seg_features =
["CreditScore","Age","Tenure","Balance","NumOfProducts","IsActiveMembe
r","EstimatedSalary"]
scaler = StandardScaler()
kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
df["Segment"] =
kmeans.fit_predict(scaler.fit_transform(df[seg_features]))
```

```
seg = df.groupby("Segment").agg(
    Count=("Exited","count"), ChurnRate=("Exited","mean"),
    AvgAge=("Age","mean"), AvgBalance=("Balance","mean"),
    ActiveRate=("IsActiveMember","mean"),
AvgProducts=("NumOfProducts","mean")
).round(2)

seg_names = {0:"Loyal actives", 1:"Multi-product", 2:"Inactive risk",
3:"Older high-risk"}
seg.index = [seg_names[i] for i in seg.index]

churned     = df[df["Exited"]==1]
not_churned = df[df["Exited"]==0]

# --- Palette -------------------------------
RED    = "#E24B4A"
AMBER  = "#EF9F27"
GREEN  = "#639922"
TEAL   = "#1D9E75"
BLUE   = "#378ADD"
GRAY   = "#888780"
BG     = "#F8F7F5"
CARD   = "#FFFFFF"
TEXT   = "#2C2C2A"
MUTED  = "#888780"

def card_ax(fig, pos, title=""):
    ax = fig.add_axes(pos)
    ax.set_facecolor(CARD)
    for sp in ax.spines.values():
        sp.set_edgecolor("#D3D1C7"); sp.set_linewidth(0.6)
    if title:
        ax.set_title(title, fontsize=9, fontweight="bold",
color=MUTED,
                     loc="left", pad=8, x=0.02)
    return ax

def churn_color(rate):
    if rate >= 0.28: return RED
    if rate >= 0.18: return AMBER
    return GREEN

# ---- Figure ------------------------------------------------
fig = plt.figure(figsize=(20, 16), facecolor=BG)
fig.text(0.02, 0.97, "Bank Customer Churn — Analysis Dashboard",
fontsize=16, fontweight="bold", color=TEXT, va="top")
fig.text(0.02, 0.945, f"10,000 customers · 20.4% churn rate · Random
Forest ROC-AUC {auc:.3f}", fontsize=10, color=MUTED, va="top")

# --- KPI cards ---------------------------------------------
```

```
kpis = [
    ("10,000", "Total customers", TEXT),
    ("20.4%",  "Churn rate",      RED),
    ("38.9 yrs","Avg age",        TEXT),
    ("$76.5k", "Avg balance",     TEXT),
    ("51.5%",  "Active members",  TEXT),
    (f"{acc*100:.0f}%",  "Model accuracy", GREEN),
    (f"{auc:.3f}", "ROC-AUC",     GREEN),
]
for i, (val, lbl, col) in enumerate(kpis):
    x0 = 0.02 + i * 0.137
    ax = card_ax(fig, [x0, 0.86, 0.125, 0.072])
    ax.text(0.5, 0.72, val,  ha="center", va="center", fontsize=14,
            fontweight="bold", color=col, transform=ax.transAxes)
    ax.text(0.5, 0.22, lbl,  ha="center", va="center", fontsize=8,
            color=MUTED, transform=ax.transAxes)
    ax.set_xticks([]); ax.set_yticks([])

# --- Feature importance
------------------------------------------------------------
ax1 = card_ax(fig, [0.02, 0.48, 0.30, 0.355], "Feature importance
(Random Forest)")
colors_fi = [RED if v > 0.18 else AMBER if v > 0.08 else GREEN for v
in fi.values]
bars = ax1.barh([feat_labels[f] for f in fi.index], fi.values,
                color=colors_fi, height=0.65, edgecolor="none")
for bar, val in zip(bars, fi.values):
    ax1.text(val + 0.003, bar.get_y() + bar.get_height()/2,
             f"{val:.1%}", va="center", fontsize=8, color=TEXT)
ax1.set_xlabel("Importance", fontsize=8, color=MUTED)
ax1.tick_params(labelsize=8, colors=TEXT)
ax1.set_xlim(0, 0.30)
ax1.grid(axis="x", alpha=0.3, linewidth=0.5)
ax1.set_facecolor(CARD)

# --- Churn by category bars
-------------------------------------------
ax2 = card_ax(fig, [0.355, 0.48, 0.295, 0.355], "Churn rate by
category")

categories = [
    ("Germany",    df[df["Geography"]=="Germany"]["Exited"].mean()),
    ("Spain",      df[df["Geography"]=="Spain"]["Exited"].mean()),
    ("France",     df[df["Geography"]=="France"]["Exited"].mean()),
    ("Female",     df[df["Gender"]=="Female"]["Exited"].mean()),
    ("Male",       df[df["Gender"]=="Male"]["Exited"].mean()),
    ("1 product",  df[df["NumOfProducts"]==1]["Exited"].mean()),
    ("2 products", df[df["NumOfProducts"]==2]["Exited"].mean()),
    ("3 products", df[df["NumOfProducts"]==3]["Exited"].mean()),
    ("4 products", df[df["NumOfProducts"]==4]["Exited"].mean()),
```

```
    ("Inactive",   df[df["IsActiveMember"]==0]["Exited"].mean()),
    ("Active",     df[df["IsActiveMember"]==1]["Exited"].mean()),
]
labels = [c[0] for c in categories]
rates  = [c[1] for c in categories]
cols   = [churn_color(r) for r in rates]

bars2 = ax2.barh(labels, rates, color=cols, height=0.65,
edgecolor="none")
for bar, val in zip(bars2, rates):
    ax2.text(val + 0.005, bar.get_y() + bar.get_height()/2,
             f"{val:.1%}", va="center", fontsize=8, color=TEXT)
ax2.axvline(0.204, color=GRAY, linewidth=1, linestyle="--", alpha=0.6)
ax2.text(0.204, len(labels)-0.3, "avg 20.4%", fontsize=7, color=MUTED)
ax2.set_xlabel("Churn rate", fontsize=8, color=MUTED)
ax2.tick_params(labelsize=8, colors=TEXT)
ax2.set_xlim(0, 1.12)
ax2.grid(axis="x", alpha=0.3, linewidth=0.5)
ax2.set_facecolor(CARD)

# ---- Separator lines between groups
for y_sep in [5.5, 7.5, 9.5]:
    ax2.axhline(y_sep, color="#D3D1C7", linewidth=0.6, linestyle="-")

# ---- Geography table
------------------------------------------------------
ax3 = card_ax(fig, [0.675, 0.66, 0.305, 0.175], "Geography breakdown")
ax3.axis("off")
geo = df.groupby("Geography").agg(
    Customers=("Exited","count"),
    AvgBalance=("Balance","mean"),
    AvgAge=("Age","mean"),
    ChurnRate=("Exited","mean")
).reset_index().sort_values("ChurnRate", ascending=False)

headers = ["Country", "Customers", "Avg balance", "Avg age", "Churn
rate"]
col_x   = [0.01, 0.22, 0.42, 0.62, 0.80]
ax3.text(0, 0.92, "", transform=ax3.transAxes)
for j, h in enumerate(headers):
    ax3.text(col_x[j], 0.88, h, transform=ax3.transAxes,
             fontsize=8, color=MUTED, fontweight="bold")

for i, row in geo.iterrows():
    y_pos = 0.65 - list(geo.index).index(i) * 0.28
    cr = row["ChurnRate"]
    cr_col = churn_color(cr)
    vals = [row["Geography"], f"{row['Customers']:,}", f"$
{row['AvgBalance']:,.0f}",
            f"{row['AvgAge']:.1f}", f"{cr:.1%}"]
```

```
    for j, v in enumerate(vals):
        col = cr_col if j == 4 else TEXT
        fw  = "bold" if j == 4 else "normal"
        ax3.text(col_x[j], y_pos, v, transform=ax3.transAxes,
                 fontsize=9, color=col, fontweight=fw)

# --- Age distribution
------------------------------------------------------------
ax4 = card_ax(fig, [0.675, 0.48, 0.305, 0.165], "Age distribution:
churned vs retained")
bins = range(18, 95, 5)
ax4.hist(not_churned["Age"], bins=bins, alpha=0.6, color=TEAL,
label="Retained",
         density=True, edgecolor="none")
ax4.hist(churned["Age"],     bins=bins, alpha=0.7, color=RED,
label="Churned",
         density=True, edgecolor="none")
ax4.axvline(churned["Age"].mean(),     color=RED,  linewidth=1.2,
linestyle="--")
ax4.axvline(not_churned["Age"].mean(), color=TEAL, linewidth=1.2,
linestyle="--")
ax4.set_xlabel("Age", fontsize=8, color=MUTED)
ax4.tick_params(labelsize=8, colors=TEXT)
ax4.legend(fontsize=8, frameon=False, labelcolor=TEXT)
ax4.set_facecolor(CARD)
ax4.grid(axis="y", alpha=0.3, linewidth=0.5)

# --- Customer segments
------------------------------------------------------------
ax5 = card_ax(fig, [0.02, 0.02, 0.96, 0.435], "Customer segments (K-
Means, 4 clusters)")
ax5.axis("off")

seg_order = seg.sort_values("ChurnRate")
n_seg = len(seg_order)
box_w = 0.22; gap = 0.02
x_starts = [0.01 + i*(box_w + gap) for i in range(n_seg)]

for i, (sname, srow) in enumerate(seg_order.iterrows()):
    x0 = x_starts[i]
    cr = srow["ChurnRate"]
    cr_col = churn_color(cr)
    rect = mpatches.FancyBboxPatch((x0, 0.05), box_w, 0.88,
        boxstyle="round,pad=0.01", linewidth=0.8,
        edgecolor="#D3D1C7", facecolor="#F8F7F5",
        transform=ax5.transAxes, clip_on=False)
    ax5.add_patch(rect)

    badge = mpatches.FancyBboxPatch((x0+0.01, 0.76), box_w-0.02,
0.155,
```

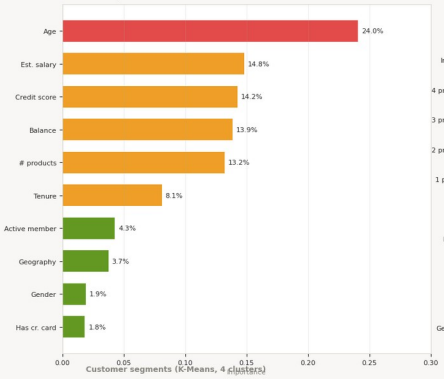
```
        boxstyle="round,pad=0.005", linewidth=0,
        edgecolor="none", facecolor=cr_col+"22",
        transform=ax5.transAxes, clip_on=False)
    ax5.add_patch(badge)

    ax5.text(x0 + box_w/2, 0.875, sname,
             transform=ax5.transAxes, ha="center", fontsize=10,
             fontweight="bold", color=cr_col)
    ax5.text(x0 + box_w/2, 0.795, f"Churn rate: {cr:.1%}",
             transform=ax5.transAxes, ha="center", fontsize=9,
color=cr_col)

    stats = [
        ("Customers",    f"{srow['Count']:,}"),
        ("Avg age",      f"{srow['AvgAge']:.1f} yrs"),
        ("Avg balance",  f"${srow['AvgBalance']:,.0f}"),
        ("Avg products", f"{srow['AvgProducts']:.2f}"),
        ("Active rate",  f"{srow['ActiveRate']:.1%}"),
    ]
    for j, (lbl, val) in enumerate(stats):
        y_pos = 0.68 - j * 0.135
        ax5.text(x0 + 0.015, y_pos, lbl,
                 transform=ax5.transAxes, fontsize=8, color=MUTED)
        ax5.text(x0 + box_w - 0.015, y_pos, val,
                 transform=ax5.transAxes, fontsize=8, color=TEXT,
                 ha="right", fontweight="bold")
        if j < len(stats)-1:
            line = plt.Line2D([x0+0.01, x0+box_w-0.01],
                              [y_pos-0.045, y_pos-0.045],
                              transform=ax5.transAxes,
color="#D3D1C7",
                              linewidth=0.5)
            ax5.add_artist(line)

#--- Legend for churn color
-------------------------------------------
ax5.text(0.94, 0.92, "Churn color key:", transform=ax5.transAxes,
         fontsize=8, color=MUTED)
for col, lbl, yy in [(GREEN,"Low (<18%)",0.83),(AMBER,"Med (18–
28%)",0.74),(RED,"High (>28%)",0.65)]:
    ax5.add_patch(mpatches.Rectangle((0.935, yy-0.005), 0.012, 0.06,
                  transform=ax5.transAxes, facecolor=col,
edgecolor="none"))
    ax5.text(0.95, yy+0.022, lbl, transform=ax5.transAxes,
fontsize=7.5, color=TEXT)
```

Bank Customer Churn — Analysis Dashboard

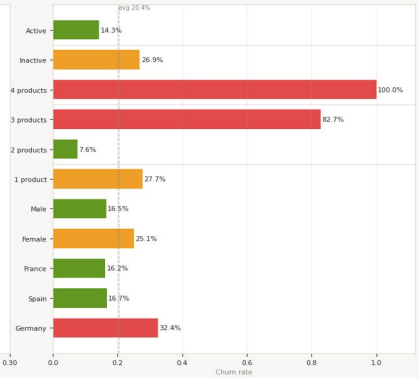
10,000 customers · 20.4% churn rate · Random Forest ROC-AUC 0.857

10,000	20.4%	38.9 yrs	$76.5k	51.5%	86%	0.857
Total customers	Churn rate	Avg age	Avg balance	Active members	Model accuracy	ROC-AUC

Feature importance (Random Forest)



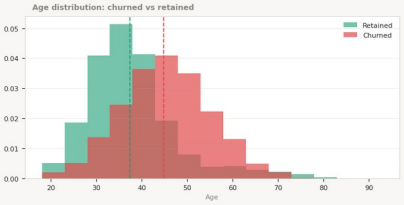
Churn rate by category



Geography breakdown

Country	Customers	Avg balance	Avg age	Churn rate
Germany	2,509	$119,730	39.8	32.4%
Spain	2,477	$61,818	38.9	16.7%
France	5,014	$62,093	38.5	16.2%

Age distribution: churned vs retained



Customer segments (K-Means, 4 clusters)

Churn color key: Low (<18%) · Med (18–28%) · High (>28%)

	Multi-product	Loyal actives	Inactive risk	Older high-risk
	Churn rate: 12.0%	Churn rate: 13.0%	Churn rate: 29.0%	Churn rate: 36.0%
Customers	2,761.0	2,876.0	3,247.0	1,116.0
Avg age	35.9 yrs	35.3 yrs	37.5 yrs	59.9 yrs
Avg balance	$9,540	$107,773	$105,903	$75,892
Avg products	2.13	1.29	1.27	1.43
Active rate	49.0%	100.0%	0.0%	83.0%